8 Module 8: Deadlocks

8.1 Banker's Algorithm - Part 1

8.1.1 Introduction to Deadlock Avoidance

- **Deadlock avoidance** is a technique used in operating systems to ensure that a system **never enters a deadlock state**.
- One of the simplest techniques for deadlock avoidance is the **resource allocation graph (RAG)** algorithm.
  - **Limitation**: The RAG algorithm works **only when all resources have a single instance**.
  - For systems with **multiple instances of resource types**, the **Banker's Algorithm** is used.

8.1.2 Overview of the Banker's Algorithm

- The Banker's Algorithm is divided into **three sub-parts**:
  1. **Introduction to the Banker's Algorithm** (covered in this lecture).
  2. **Safety Algorithm** (to be covered in the next session).
  3. **Resource Request Algorithm** (to be covered later).

8.1.2.1 Purpose of the Banker's Algorithm

- Designed to **avoid deadlocks** by carefully allocating resources to processes such that the system remains in a **safe state**.
- **Analogy**: Named after a banking system where the bank **never allocates available cash** in a way that could prevent it from satisfying the needs of all customers.
  - Similarly, the OS ensures that resource allocation does not lead to a state where processes are indefinitely blocked.

8.1.2.2 Key Steps in the Banker's Algorithm

1. **Initialization of Data Structures** (to encode the state of resource allocation).
2. **Resource Request** (a process requests resources).
3. **Safety Check** (the system verifies if allocation keeps the system in a safe state).
4. **Resource Allocation** (if safe, resources are allocated; otherwise, the process waits).
5. **Resource Release** (processes return allocated resources after completion in finite time).

8.1.3 Data Structures in the Banker's Algorithm

To implement the Banker's Algorithm, **four key data structures** are required. Let: - **n** = number of processes. - **m** = number of resource types.

8.1.3.1 1. Available Vector

- **Definition**: A vector of length **m** that indicates the number of **available resources** of each type.
- **Notation**: `Available[j] = k` means there are **k instances** of resource type **R_j** currently available.
- **Example**: If `Available = [3, 3, 2]`, then:
  - 3 instances of **Resource A** are available.
  - 3 instances of **Resource B** are available.
  - 2 instances of **Resource C** are available.

8.1.3.2 2. Max Matrix

- **Definition**: An **n × m** matrix that defines the **maximum demand** of each process.

- **Notation**: `Max[i][j] = k` means process **P_i** may request **at most k instances** of resource type **R_j**.
- **Example**:
  - If `Max[0][0] = 7`, process **P_0** may request up to **7 instances of Resource A**.
  - The **total demand** of a process cannot exceed the **total resources** in the system.

8.1.3.3 3. Allocation Matrix

- **Definition**: An **n × m** matrix that defines the number of resources **currently allocated** to each process.
- **Notation**: `Allocation[i][j] = k` means process **P_i** is currently holding **k instances** of resource type **R_j**.
- **Example**:
  - If `Allocation[0][1] = 1`, process **P_0** holds **1 instance of Resource B**.

8.1.3.4 4. Need Matrix

- **Definition**: An **n × m** matrix that indicates the **remaining resource needs** of each process.
- **Calculation**: `Need[i][j] = Max[i][j] - Allocation[i][j]`.
- **Notation**: `Need[i][j] = k` means process **P_i** may still need **k more instances** of **R_j** to complete.
- **Example**:
  - If `Max[0][0] = 7` and `Allocation[0][0] = 0`, then `Need[0][0] = 7`.

8.1.4 Example: System State at Time T0

Consider a system with: - **5 processes**: P0, P1, P2, P3, P4. - **3 resource types**: A (10 instances), B (5 instances), C (7 instances).

8.1.4.1 Given Matrices at T0

Process	**Max Matrix (A, B, C)**	**Allocation Matrix (A, B, C)**
P0	(7, 5, 3)	(0, 1, 0)
P1	(3, 2, 2)	(2, 0, 0)
P2	(9, 0, 2)	(3, 0, 2)
P3	(2, 2, 2)	(2, 1, 1)
P4	(4, 3, 3)	(0, 0, 2)

8.1.4.2 Step 1: Calculate Total Allocated Resources

- Sum the **Allocation Matrix** column-wise:
  - **Resource A**: 0 + 2 + 3 + 2 + 0 = **7**
  - **Resource B**: 1 + 0 + 0 + 1 + 0 = **2**
  - **Resource C**: 0 + 0 + 2 + 1 + 2 = **5**
- **Total Allocated**: (7, 2, 5)

8.1.4.3 Step 2: Calculate Available Resources

- **Total Resources**: (10, 5, 7)
- **Available = Total - Allocated**:
  - **A**: 10 - 7 = **3**
  - **B**: 5 - 2 = **3**
  - **C**: 7 - 5 = **2**
- **Available Vector**: (3, 3, 2)

8.1.4.4 Step 3: Calculate Need Matrix

Using `Need = Max - Allocation` (element-wise subtraction):

Process	**Need Matrix (A, B, C)**
P0	(7-0, 5-1, 3-0) = (7, 4, 3)
P1	(3-2, 2-0, 2-0) = (1, 2, 2)
P2	(9-3, 0-0, 2-2) = (6, 0, 0)
P3	(2-2, 2-1, 2-1) = (0, 1, 1)
P4	(4-0, 3-0, 3-2) = (4, 3, 1)

8.1.4.5 Summary of Calculations

- **Available**: (3, 3, 2)
- **Need Matrix**:
  - P0: (7, 4, 3)
  - P1: (1, 2, 2)
  - P2: (6, 0, 0)
  - P3: (0, 1, 1)
  - P4: (4, 3, 1)

8.1.5 Key Takeaways

1. **Data Structures**:
   - **Available**: Tracks free resources.
   - **Max**: Maximum demand per process.
   - **Allocation**: Currently held resources per process.
   - **Need**: Remaining resources required per process.
2. **Calculations**:
   - **Available = Total Resources - Allocated Resources**.
   - **Need = Max - Allocation**.
3. **Safe State**:
   - The system must ensure that resource allocation keeps the system in a **safe state** (to be explored in the next lecture on the **Safety Algorithm**).

8.2 Banker's Algorithm - Part 2

8.2.1 1. Introduction to the Safety Algorithm

- The **Safety Algorithm** is a key component of the **Banker's Algorithm**, used to determine whether a system is in a **safe state**.
- A system is in a **safe state** if there exists at least one sequence of process executions where:
  - Each process can obtain its **maximum required resources**.
  - Each process can **complete its execution**.
  - Each process **releases its resources** after completion, allowing subsequent processes to proceed without deadlock.

8.2.2 2. Steps of the Safety Algorithm

The algorithm proceeds in **four main steps**:

8.2.2.1 Step 1: Initialization

- Define two vectors:
  - **Work** (length = *m*, where *m* = number of resource types)
    - Initialized as: **Work = Available** (current available resources in the system).
  - **Finish** (length = *n*, where *n* = number of processes)
    - Initialized as: **Finish[i] = false** for all processes *i* (0 <= *i* <= *n-1*).
    - Indicates that **no process has finished execution** at the start.

8.2.2.2 Step 2: Find an Eligible Process

- Search for a process *i* that satisfies **both** of the following conditions:
  1. **Finish[i] == false**
     - The process has **not yet finished**.
  2. **Need[i] <= Work**
     - The **resources needed** by process *i* are **<= currently available resources (Work)**.
     - **Need[i]** is computed as: **Need[i] = Max[i] - Allocation[i]**.
- If **no such process *i* exists**, proceed to **Step 4**.
- If a process *i* satisfies both conditions, proceed to **Step 3**.

8.2.2.3 Step 3: Resource Allocation Simulation

- **Simulate process *i* completing and releasing its resources**:
  - Update **Work**:
    - **Work = Work + Allocation[i]**
    - This represents the **release of resources** by process *i* after completion.
  - Update **Finish[i] = true**
    - Mark process *i* as **finished**.
  - **Repeat from Step 2** to check for the next eligible process.

8.2.2.4 Step 4: Check for Safe State

- If **Finish[i] == true for all processes *i*** (0 <= *i* <= *n-1*):
  - The system is in a **safe state**.
  - A **safe sequence** exists where all processes can complete without deadlock.
- If **any Finish[i] == false** after exhaustive checks:
  - The system is in an **unsafe state** (potential deadlock).

8.2.3 3. Example: Safety Algorithm Execution

8.2.3.1 System Configuration

- **5 processes (P0, P1, P2, P3, P4)**
- **3 resource types (R0, R1, R2)**
- **Initial state at time T0**:
  - **Available = [3, 3, 2]**
  - **Allocation, Max, and Need matrices** are provided (implied but not explicitly shown in the transcript).

8.2.3.2 Step-by-Step Execution

8.2.3.2.1 Initialization (Step 1)

- **Work = Available = [3, 3, 2]**
- **Finish = [F, F, F, F, F]** (F = False)
- **Safe Sequence = []** (initially empty)

8.2.3.2.2 First Iteration (Step 2: Find Eligible Process)

- **Check P0 (i=0):**
  - Finish[0] == false? **Yes**
  - Need[0] <= Work? **No** (element-wise comparison fails)
  - **P0 must wait**.
- **Check P1 (i=1):**
  - Finish[1] == false? **Yes**
  - Need[1] <= Work? **Yes** (satisfies condition)
  - **P1 is eligible**.

8.2.3.2.3 Resource Allocation for P1 (Step 3)

- **Work = Work + Allocation[1] = [3,3,2] + [2,0,0] = [5,3,2]**
- **Finish[1] = true**
- **Safe Sequence = [P1]**

8.2.3.2.4 Check P2 (i=2):

- Finish[2] == false? **Yes**
- Need[2] <= Work? **No**
- **P2 must wait**.

8.2.3.2.5 Check P3 (i=3):

- Finish[3] == false? **Yes**
- Need[3] <= Work? **Yes**
- **P3 is eligible**.

8.2.3.2.6 Resource Allocation for P3 (Step 3)

- **Work = [5,3,2] + [2,1,1] = [7,4,3]**
- **Finish[3] = true**
- **Safe Sequence = [P1, P3]**

8.2.3.2.7 Check P4 (i=4):

- Finish[4] == false? **Yes**
- Need[4] <= Work? **Yes**
- **P4 is eligible**.

8.2.3.2.8 Resource Allocation for P4 (Step 3)

- **Work = [7,4,3] + [0,0,2] = [7,4,5]**
- **Finish[4] = true**
- **Safe Sequence = [P1, P3, P4]**

8.2.3.2.9 End of First Round

- **Finish = [F, T, F, T, T]**
  - Not all processes are finished (**P0 and P2 remain**).
  - **Proceed to next iteration**.

8.2.3.2.10 Second Iteration (Step 2: Recheck Processes)

- **Check P0 (i=0):**
  - Finish[0] == false? **Yes**
  - Need[0] <= Work? **Yes** (now satisfies condition)
  - **P0 is eligible**.

8.2.3.2.11 Resource Allocation for P0 (Step 3)

- **Work = [7,4,5] + [0,1,0] = [7,5,5]**
- **Finish[0] = true**
- **Safe Sequence = [P1, P3, P4, P0]**

8.2.3.2.12 Check P1 (i=1):

- Finish[1] == false? **No** (already finished)
- **Skip**.

8.2.3.2.13 Check P2 (i=2):

- Finish[2] == false? **Yes**
- Need[2] <= Work? **Yes**
- **P2 is eligible**.

8.2.3.2.14 Resource Allocation for P2 (Step 3)

- **Work = [7,5,5] + [3,0,2] = [10,5,7]**
- **Finish[2] = true**
- **Safe Sequence = [P1, P3, P4, P0, P2]**

8.2.3.2.15 Check P3 and P4 (i=3,4):

- Both are already finished (**Finish[3] = Finish[4] = true**).
- **Skip**.

8.2.3.2.16 Final Check (Step 4)

- **Finish = [T, T, T, T, T]**
  - **All processes are finished**.
  - **System is in a safe state**.

8.2.3.3 Final Safe Sequence

- **Order of execution to avoid deadlock: P1 -> P3 -> P4 -> P0 -> P2**

8.2.4 4. Key Observations

1. **Safe State vs. Unsafe State**:
   - A **safe state** guarantees that **at least one sequence** exists where all processes can complete.
   - An **unsafe state** does **not** guarantee deadlock but indicates a **potential risk**.
2. **Role of the Work Vector**:
   - **Work** dynamically updates to reflect **available resources + released resources** from finished processes.
3. **Need Matrix Importance**:
   - **Need[i] = Max[i] - Allocation[i]**
   - Determines whether a process can proceed with the **currently available resources**.
4. **Termination Condition**:
   - The algorithm terminates when:
     - **All processes are marked finished (safe state)**.
     - **No more processes can be allocated (unsafe state)**.

8.2.5 5. Summary

- The **Safety Algorithm** verifies whether a system is in a **safe state** by simulating process execution and resource release.
- It uses:
  - **Work vector** (available resources).
  - **Finish vector** (process completion status).
  - **Need matrix** (remaining resource requirements).
- A **safe sequence** is derived if all processes can complete without deadlock.
- The next session will cover the **Resource Allocation Algorithm**, which uses the Safety Algorithm to make **real-time allocation decisions**.

8.3 Banker's Algorithm - Part 3

8.3.1 Introduction to Resource Request Algorithm

- The **Resource Request Algorithm** is a critical component of the **Banker's Algorithm**.
- It handles **resource requests** from processes while ensuring the system remains in a **safe state**.
- The algorithm evaluates whether a request can be granted without leading to a **deadlock**.

8.3.2 Resource Request Algorithm: Overview

- When a process **Pi** makes a request for resources, it is represented by a **request vector**.
  - If **$Request_i = k$**, it means process **Pi** wants **k** instances of resource type **R_j**.
- The algorithm follows **three key steps** to determine whether the request can be granted.

8.3.3 Three Steps of the Resource Request Algorithm

8.3.3.1 Step 1: Check Against Maximum Claim (Need Matrix)

- The algorithm verifies whether the **requested resources** are within the process's **declared maximum needs**.
  - Condition: **Requesti <= Needi**
    - If **True** -> Proceed to **Step 2**.
    - If **False** -> **Error condition** (process has exceeded its maximum claim).

8.3.3.2 Step 2: Check Resource Availability

- The algorithm checks if the **requested resources** are **currently available**.
  - Condition: **Requesti <= Available**
    - If **True** -> Proceed to **Step 3**.
    - If **False** -> **Process Pi must wait** (resources not available).

8.3.3.3 Step 3: Pretend Allocation & Safety Check

- The algorithm **temporarily allocates** the requested resources to **Pi** by updating the system state:
  - **Available = Available - Requesti**
  - **Allocationi = Allocationi + Requesti**
  - **Needi = Needi - Requesti**
- The **Safety Algorithm** is then executed to check if the **new state is safe**.
  - If **safe** -> **Permanently allocate** resources to **Pi**.
  - If **unsafe** -> **Roll back** the temporary allocation, and **Pi must wait**.

8.3.4 Example: Applying the Resource Request Algorithm

8.3.4.1 System Configuration

- **5 processes (P0 to P4)**
- **3 resource types (A, B, C)**
- Initial state at **T = T0** (data structures: **Available, Allocation, Need**).

8.3.4.2 Request Scenario

- **Process P1** makes a request:
  - **Request vector = (1, 0, 2)** (1 instance of A, 0 of B, 2 of C).

8.3.4.3 Step-by-Step Execution

8.3.4.3.1 Step 1: Check Against Need Vector

- **Need vector for P1 = (1, 2, 2)**
- **Request vector = (1, 0, 2)**
- **Condition:** (1 <= 1) **and** (0 <= 2) **and** (2 <= 2) -> **True** (proceed to Step 2).

8.3.4.3.2 Step 2: Check Against Available Vector

- **Available vector = (3, 3, 2)**
- **Request vector = (1, 0, 2)**
- **Condition:** (1 <= 3) **and** (0 <= 3) **and** (2 <= 2) -> **True** (proceed to Step 3).

8.3.4.3.3 Step 3: Pretend Allocation & Safety Check

- **Update system state:**
  - **Available = (3-1, 3-0, 2-2) = (2, 3, 0)**
  - **Allocation (P1) = Previous Allocation + (1, 0, 2)**
  - **Need (P1) = Previous Need - (1, 0, 2) = (0, 2, 0)**
- **Run Safety Algorithm** to verify if the new state is safe.
  - If **safe** -> Allocate resources to **P1**.
  - If **unsafe** -> Revert changes, **P1 waits**.

8.3.5 Summary of Key Concepts

1. **Resource Request Algorithm** is triggered when a process requests resources.
2. **Three-step verification:**
   - Check if request <= declared maximum (**Need**).
   - Check if request <= **Available** resources.
   - **Pretend allocation** and run **Safety Algorithm**.
3. **If safe** -> Grant resources.
4. **If unsafe** -> **Roll back** and make the process wait.

8.3.6 Conclusion

- The **Resource Request Algorithm** ensures that **resource allocation** does not lead to an **unsafe state**.
- It works in conjunction with the **Safety Algorithm** to maintain **deadlock avoidance**.
- **Key takeaway:** Always verify a request using the **three-step process** before granting resources.

8.4 Deadlock Avoidance - An Introduction

8.4.1 1. Introduction to Deadlock Avoidance

- **Context**:
  - Deadlock prevention focuses on ensuring that **at least one** of the four necessary conditions for deadlock (**mutual exclusion, hold and wait, no preemption, circular wait**) does not hold.
  - Deadlock **avoidance** is an alternative technique that dynamically assesses resource allocation to prevent deadlocks.
- **Core Idea**:
  - The **operating system (OS)** makes decisions about whether a process should **wait** or **proceed** based on **prior knowledge** of all future resource requests and releases.
  - Unlike prevention, avoidance does **not** restrict resource access arbitrarily but instead **analyzes system state** before allocation.
- **Advantages**:
  - **Simplicity**: The algorithm is straightforward in principle.
- **Challenges**:
  - **Prediction Difficulty**: Requires **complete a priori knowledge** of all process resource requests and releases, which is often impractical in real-world systems.

8.4.2 2. Resource Allocation State

The **resource allocation state** of a system is defined by three key components: 1. **Number of Available Resources** (`Available`): - Resources currently **not allocated** to any process. 2. **Number of Allocated Resources** (`Allocation`): - Resources **currently held** by each process. 3. **Maximum Demands of Processes** (`Max`): - The **peak resource requirement** each process may request during execution.

- **Purpose**:
  - The OS uses this information to **determine whether granting a resource request** will lead to a **safe** or **unsafe** state.

8.4.3 3. Safe State vs. Unsafe State

8.4.3.1 3.1 Definition of a Safe State

- A system is in a **safe state** if there exists **at least one sequence** of process executions (`<P1, P2, ..., Pn>`) such that:

1. Each process `Pi` can **obtain its maximum required resources** (either immediately or after waiting).
2. After `Pi` completes, it **releases all allocated resources**, making them available for subsequent processes.
3. This sequence **guarantees that all processes can terminate** without deadlock.

- **Example**:
  - If `P1` needs 3 units of a resource but only 2 are available, `P1` must wait until another process (`P2`) finishes and releases its resources.
  - Once `P2` completes, `P1` can acquire the needed resources, execute, and terminate, allowing `P3` to proceed, and so on.

8.4.3.2 3.2 Implications of Safe vs. Unsafe States

State	Definition	Deadlock Risk
Safe State	A sequence exists where all processes can complete without deadlock.	**No deadlock possible**.
Unsafe State	No such sequence exists; processes may block indefinitely.	**Deadlock may (but not always) occur**.

- **Key Observation**:
  - **All deadlocks occur in unsafe states**, but **not all unsafe states lead to deadlock**.
  - **Avoidance Goal**: Ensure the system **never enters an unsafe state**.

8.4.4 4. Deadlock Avoidance Mechanism

8.4.4.1 4.1 Process Flow

1. A process **requests a resource**.
2. The OS **checks whether allocating the resource** would keep the system in a **safe state**.
   - If **yes** -> Allocate the resource.
   - If **no** -> The process **must wait** (avoiding potential deadlock).

8.4.4.2 4.2 Techniques for Avoidance The choice of technique depends on the **resource instance count**: 1. **Single Instance of a Resource Type**: - Use a **Resource Allocation Graph (RAG)** with **claim edges** (dashed lines representing future requests). - The OS simulates allocations to detect **potential cycles** (indicating unsafe states). 2. **Multiple Instances of a Resource Type**: - Use the **Banker's Algorithm** (discussed in subsequent lectures). - The algorithm dynamically checks if the system remains in a safe state after each allocation.

8.4.5 5. Summary of Key Concepts

1. **Deadlock Avoidance**:
   - Requires **complete prior knowledge** of all process resource requests and releases.
   - **Simple in theory** but **challenging in practice** due to prediction difficulties.
2. **Resource Allocation State**:
   - Defined by **available resources**, **allocated resources**, and **maximum process demands**.
3. **Safe State**:
   - A state where **all processes can complete** in some order without deadlock.
   - **No deadlock possible** if the system remains in a safe state.
4. **Unsafe State**:
   - **No guaranteed completion sequence** exists; deadlock **may** (but not necessarily) occur.
5. **Avoidance Strategies**:

- **Single-instance resources**: Resource Allocation Graph.
- **Multi-instance resources**: Banker's Algorithm.

8.5 Deadlock Characterization

8.5.1 Introduction to Deadlock Characterization

- Before addressing how to **handle deadlocks**, it is essential to understand the **characteristics** that define a deadlock.
- A deadlock occurs **only if four necessary conditions** are **simultaneously** present in a system:
  1. **Mutual Exclusion**
  2. **Hold and Wait**
  3. **No Preemption**
  4. **Circular Wait**
- These conditions are **collectively required** for a deadlock to manifest.

8.5.2 1. Mutual Exclusion

8.5.2.1 Definition

- **Mutual exclusion** ensures that **only one process** can use a **resource** at any given time.
- It is enforced to **prevent race conditions**, which occur when multiple processes access shared data concurrently, leading to unpredictable outcomes.

8.5.2.2 When is Mutual Exclusion Enforced?

- Mutual exclusion is **required** when a resource is **non-sharable**.
  - If a resource is **sharable** (e.g., read-only files), multiple processes can access it simultaneously **without mutual exclusion**.
  - If a resource is **non-sharable** (e.g., a printer, a file being modified), only **one process** can use it at a time, and others must **wait** until it is released.

8.5.2.3 Example Scenario

- **Diagram Analysis**:
  - If **Resource 1** and **Resource 2** are **sharable**, both **Process 1** and **Process 2** can use them **simultaneously** -> **no deadlock**.
  - If **Resource 1** and **Resource 2** are **non-sharable**:
    - **Process 1** holds **Resource 1** and requests **Resource 2**.
    - **Process 2** holds **Resource 2** and requests **Resource 1**.
    - Both processes **wait indefinitely** -> **deadlock occurs due to mutual exclusion**.

8.5.2.4 Key Takeaway

- Mutual exclusion is **not inherently problematic**-it is **necessary** for non-sharable resources.
- However, when combined with other conditions, it **contributes to deadlocks**.

8.5.3 2. Hold and Wait

8.5.3.1 Definition

- A process is in a **hold-and-wait** state if:
  - It **holds at least one resource** (e.g., **Resource 1**).

- It is **simultaneously waiting** to acquire **additional resources** held by other processes (e.g., **Resource 2**).

8.5.3.2 Example Scenario

- **Process 1**:
  - Holds **Resource 1**.
  - Requests **Resource 2** (held by **Process 2**).
- **Process 2**:
  - Holds **Resource 2**.
  - Requests **Resource 1** (held by **Process 1**).
- Both processes are **blocked**, waiting for each other -> **deadlock**.

8.5.3.3 Key Takeaway

- The **hold-and-wait** condition arises when processes **do not release** their current resources while **requesting new ones**.
- This condition **alone does not cause deadlocks** but is **essential** for deadlock formation when combined with other conditions.

8.5.4 3. No Preemption

8.5.4.1 Definition

- **No preemption** means that a resource **cannot be forcibly taken** from a process.
- A resource is **only released voluntarily** by the process holding it **after completing its task**.

8.5.4.2 Example Scenario

- **Process 1** holds **Resource 1** and requests **Resource 2** (held by **Process 2**).
- **Process 2** holds **Resource 2** and requests **Resource 1** (held by **Process 1**).
- If **preemption were allowed**:
  - The **operating system** could **forcibly take** a resource from one process and **assign it to another**.
  - Example: If **Process 2** releases **Resource 2** (voluntarily or forcibly), **Process 1** could proceed, complete its task, and release **Resource 1**, allowing **Process 2** to continue.
- Since **no preemption** is enforced:
  - Resources are **only released when the holding process finishes**.
  - Both processes **remain blocked** -> **deadlock persists**.

8.5.4.3 Key Takeaway

- The **absence of preemption** ensures that processes **retain resources** until they **voluntarily release** them.
- This condition **prevents external intervention** from resolving the deadlock.

8.5.5 4. Circular Wait

8.5.5.1 Definition

- A **circular wait** occurs when there is a **cyclic dependency** among processes.
- Formally, a set of processes **{P0, P1, P2, …, Pn}** exists where:
  - **P0** is waiting for a resource held by **P1**.
  - **P1** is waiting for a resource held by **P2**.
  - …

  - **Pn-1** is waiting for a resource held by **Pn**.
  - **Pn** is waiting for a resource held by **P0**.
- This forms a **closed loop**, preventing any process from progressing.

8.5.5.2 Example Scenario

- **Process 1** holds **Resource 1** and waits for **Resource 2** (held by **Process 2**).
- **Process 2** holds **Resource 2** and waits for **Resource 1** (held by **Process 1**).
- The **circular dependency** ensures that **neither process can proceed** -> **deadlock**.

8.5.5.3 Key Takeaway

- **Circular wait** is the **final condition** that **completes the deadlock cycle**.
- Without this condition, the other three (**mutual exclusion, hold-and-wait, no preemption**) may exist **without causing a deadlock**.

8.5.6 Summary of Deadlock Conditions

Condition	Description	Role in Deadlock
Mutual Exclusion	Only one process can use a resource at a time (enforced for non-sharable resources).	Ensures processes **compete** for resources.
Hold and Wait	A process holds a resource while waiting for another.	Creates **dependency** between processes.
No Preemption	Resources cannot be forcibly taken; only released voluntarily.	Prevents **external resolution** of deadlocks.
Circular Wait	A cyclic chain of processes exists where each waits for a resource held by another.	**Final condition** that **locks processes** in a deadlock.

8.5.6.1 Critical Observation

- **All four conditions must hold simultaneously** for a deadlock to occur.
- If **any one condition is broken**, the deadlock **can be prevented or resolved**.

8.5.7 Conclusion

- Deadlocks are a **serious issue** in operating systems, leading to **system hangs** and **resource wastage**.
- Understanding the **four necessary conditions** (**mutual exclusion, hold-and-wait, no preemption, circular wait**) is **fundamental** to:
  - **Detecting** deadlocks.
  - **Preventing** deadlocks by breaking one or more conditions.
  - **Recovering** from deadlocks when they occur.
- Future lectures will explore **strategies** to **handle deadlocks** based on these conditions.

8.6 Methods of Handling Deadlock

8.6.1 Introduction to Deadlock Handling

- Deadlocks are a critical issue in operating systems where processes are permanently blocked due to resource contention.
- This lecture explores **three primary methods** for handling deadlocks:

1. **Deadlock Prevention & Avoidance** (Proactive approaches)
2. **Deadlock Detection & Recovery** (Reactive approach)
3. **Deadlock Ignorance** (No explicit handling)

8.6.2 1. Deadlock Prevention & Avoidance

- Both methods **ensure the system never enters a deadlock state**.
- **Key difference**:
  - **Prevention** eliminates one of the **four necessary conditions** for deadlock.
  - **Avoidance** uses **prior knowledge** of process resource requests to make safe allocation decisions.

8.6.2.1 1.1 Deadlock Prevention

- **Goal**: Ensure **at least one** of the four necessary conditions for deadlock **cannot occur**.
- **Four Necessary Conditions for Deadlock** (Coffman Conditions):
  1. **Mutual Exclusion**: Only one process can use a resource at a time.
  2. **Hold and Wait**: A process holds a resource while waiting for another.
  3. **No Preemption**: Resources cannot be forcibly taken from a process.
  4. **Circular Wait**: A circular chain of processes exists, each waiting for a resource held by another.
- **Prevention Strategy**: Break **one or more** of these conditions.
- **No prior knowledge** of process resource usage is required.

8.6.2.2 1.2 Deadlock Avoidance

- **Goal**: Use **advance information** about process resource requests to **prevent unsafe states**.
- **Key Mechanism**:
  - The OS **collects information** about future resource requests from processes.
  - Before granting a request, the OS **checks if allocation leads to a safe state**.
  - If **safe**, the request is granted; otherwise, it is **denied or delayed**.
- **Requires prior knowledge** of process resource needs (e.g., maximum claims).
- **Example**: Banker's Algorithm (Dijkstra) is a classic avoidance technique.

8.6.2.3 Comparison: Prevention vs. Avoidance

Aspect	Deadlock Prevention	Deadlock Avoidance
Approach	Eliminates a deadlock condition.	Uses resource request info to avoid deadlock.
Prior Knowledge	Not required.	Required (processes declare max needs).
Flexibility	Restrictive (may limit concurrency).	More adaptive (allows safe allocations).
Overhead	Lower (no dynamic checks).	Higher (requires state tracking).

8.6.3 2. Deadlock Detection & Recovery

- **Goal**: **Allow deadlocks to occur**, then **detect and recover** from them.
- **Two Phases**:
  1. **Detection**: Periodically check the system for deadlocks.
  2. **Recovery**: Apply corrective measures once a deadlock is found.
- **Used in systems** where deadlocks are **rare** or prevention/avoidance is **too costly**.
- **Detection Methods**:
  - **Resource Allocation Graph (RAG)**: Detects cycles (indicating deadlock).

  - **Wait-for Graph (WFG)**: Simplified graph for detection.
- **Recovery Strategies**:
  1. **Process Termination**:
     - Kill **all deadlocked processes**.
     - Kill **one process at a time** until the deadlock resolves.
  2. **Resource Preemption**:
     - **Rollback**: Preempt resources from processes and allocate them to others.
     - **Starvation Prevention**: Ensure the same process is not always preempted.

8.6.4 3. Deadlock Ignorance (Ostrich Algorithm)

- **Goal**: **Pretend deadlocks do not exist** and take no explicit action.
- **Rationale**:
  - Deadlocks are **rare** in many systems.
  - Prevention/avoidance/detection introduce **overhead**.
  - **Modern OS (UNIX, Windows, Linux)** often use this approach.
- **Handling Responsibility**:
  - **Application developers** must implement their own deadlock handling (e.g., timeouts, lock ordering).
  - **Example**: Database systems often include **application-level deadlock detection**.
- **Trade-offs**:
  - **Pros**: Simplicity, low overhead.
  - **Cons**: Deadlocks may **crash applications** or require manual intervention.

8.6.5 Summary of Deadlock Handling Methods

Method	Description	Pros	Cons	Examples
Prevention	Eliminates one of the four deadlock conditions.	Guarantees no deadlocks.	Reduces concurrency; restrictive.	Mutual exclusion removal.
Avoidance	Uses prior resource info to avoid unsafe states.	More flexible than prevention.	Requires process cooperation; overhead.	Banker's Algorithm.
Detection & Recovery	Lets deadlocks occur, then detects and fixes them.	No upfront restrictions.	Recovery may be costly (e.g., killing processes).	UNIX (early versions).
Ignorance	Assumes deadlocks won't happen; leaves handling to applications.	Low overhead; simple.	Deadlocks may cause crashes.	Windows, Linux, Modern UNIX.

8.6.6 Key Takeaways

1. **Proactive Methods (Prevention & Avoidance)**:
   - **Prevention** breaks deadlock conditions **without prior knowledge**.
   - **Avoidance** uses **resource request info** to make safe decisions.
2. **Reactive Method (Detection & Recovery)**:
   - Allows deadlocks but **fixes them after occurrence**.
3. **Passive Method (Ignorance)**:
   - **No OS-level handling**; applications must manage deadlocks.

4. **Real-World Usage**:
    - **Prevention/Avoidance**: Used in **critical systems** (e.g., aerospace, real-time OS).
    - **Detection/Recovery**: Used in **legacy systems** (e.g., older UNIX).
    - **Ignorance**: Dominant in **modern OS** (Windows, Linux), relying on **application-level solutions**.

8.7 Multiple Instances of Each Resource Type

8.7.1 Introduction to Deadlock Detection with Multiple Resource Instances

- **Motivation**:
  - Deadlock detection is essential to prevent system freezes and ensure efficient resource utilization.
  - Traditional **weighted graph schemes** (e.g., Wait-For Graphs) are limited to **single instances of each resource type**.
  - Many real-world systems have **multiple instances of each resource type**, rendering the weighted graph scheme inadequate.
  - A **more sophisticated algorithm** is required to handle deadlock detection in such scenarios.

8.7.2 Data Structures for Deadlock Detection

The algorithm employs data structures similar to those used in the **Banker's Algorithm**: - **Definitions**: - **n**: Number of processes in the system. - **m**: Number of resource types. - **Key Data Structures**: 1. **Available Vector** (`Available`): - Length: `m`. - Purpose: Indicates the number of **available resources** of each type. 2. **Allocation Matrix** (`Allocation`): - Dimensions: `n × m`. - Purpose: Defines the number of resources of **each type currently allocated** to each process. 3. **Request Matrix** (`Request`): - Dimensions: `n × m`. - Purpose: Indicates the **current request** of each process for each resource type.

8.7.3 Deadlock Detection Algorithm

8.7.3.1 Step 1: Initialization

- **Work Vector** (`Work`):
  - Initialized to `Available` (i.e., `Work = Available`).
- **Finish Vector** (`Finish`):
  - Length: `n`.
  - Initialization:
    * For each process `P_i`:
      · If `Allocation_i != 0` (i.e., process holds at least one resource), set `Finish[i] = false`.
      · Else, set `Finish[i] = true`.

8.7.3.2 Step 2: Resource Reclamation and Process Completion

- **Objective**: Find a process `P_i` that can **potentially complete** and release its resources.
- **Condition**:
  - Find an index `i` such that:
    1. `Finish[i] = false` (process has not finished).
    2. `Request_i <= Work` (process's request can be satisfied with available resources).
- **Actions if Condition is Met**:
  1. Update `Work = Work + Allocation_i` (reclaim resources from `P_i`).
  2. Set `Finish[i] = true` (mark `P_i` as completed).
  3. **Repeat Step 2** (check for other processes).
- **Actions if No Such `i` Exists**:
  - Proceed to **Step 4** (deadlock check).

8.7.3.3 Step 3: Optimistic Assumption

- **Why Reclaim Resources Early?**
  - The algorithm **assumes optimistically** that `P_i` will:
    - Not request additional resources.
    - Release all allocated resources soon.
  - **Implication**:
    - If the assumption is incorrect, a deadlock may occur later, but it will be detected in the **next run** of the algorithm.

8.7.3.4 Step 4: Deadlock Detection

- **Check Finish Vector**:
  - If **any** `Finish[i] = false`, the system is in a **deadlock state**.
  - The corresponding process `P_i` is **deadlocked**.

8.7.4 Example Walkthrough

8.7.4.1 System Configuration

- **Processes**: `P_0, P_1, P_2, P_3, P_4` (5 processes).
- **Resource Types**: `A, B, C` (3 types).
- **Instances**:
  - `A`: 7 instances.
  - `B`: 2 instances.
  - `C`: 6 instances.
- **Initial State**:
  - `Available = [0, 0, 0]`.
  - `Allocation` and `Request` matrices are provided (not explicitly listed in transcript but inferred from steps).

8.7.4.2 Initialization

- `Work = Available = [0, 0, 0]`.
- `Finish = [false, false, false, false, false]` (since all processes hold resources).

8.7.4.3 Iteration 1: Process `P_0` (i = 0)

- **Check Conditions**:
  - `Finish[0] = false` [OK]
  - `Request_0 = [0, 0, 0] <= Work = [0, 0, 0]` [OK]
- **Actions**:
  - `Work = Work + Allocation_0` (updated).
  - `Finish[0] = true`.

8.7.4.4 Iteration 2: Process `P_1` (i = 1)

- **Check Conditions**:
  - `Finish[1] = false` [OK]
  - `Request_1 > Work` [X] (cannot be satisfied).
- **Result**: Skip `P_1`.

8.7.4.5 Iteration 3: Process `P_2` (i = 2)

- **Check Conditions**:
  - `Finish[2] = false` [OK]
  - `Request_2 <= Work` [OK]
- **Actions**:
  - `Work = Work + Allocation_2`.
  - `Finish[2] = true`.

8.7.4.6 Iteration 4: Process `P_3` (i = 3)

- **Check Conditions**:
  - `Finish[3] = false` [OK]
  - `Request_3 <= Work` [OK]
- **Actions**:
  - `Work = Work + Allocation_3`.
  - `Finish[3] = true`.

8.7.4.7 Iteration 5: Process `P_4` (i = 4)

- **Check Conditions**:
  - `Finish[4] = false` [OK]
  - `Request_4 <= Work` [OK]
- **Actions**:
  - `Work = Work + Allocation_4`.
  - `Finish[4] = true`.

8.7.4.8 Recheck Process `P_1` (i = 1)

- **Check Conditions**:
  - `Finish[1] = false` [OK]
  - `Request_1 <= Work` [OK] (now satisfiable due to reclaimed resources).
- **Actions**:
  - `Work = Work + Allocation_1`.
  - `Finish[1] = true`.

8.7.4.9 Final State

- `Finish = [true, true, true, true, true]`.
- **Conclusion**: **No deadlock** exists.

8.7.4.10 Modified Scenario: Additional Request by `P_2`

- **Change**: `P_2` requests an **additional instance of resource** `C`.
- **Result**:
  - Only `P_0` can be satisfied initially.
  - `P_1, P_2, P_3, P_4` cannot be satisfied (`Request_i > Work`).
  - **Deadlock detected** involving `P_1, P_2, P_3, P_4`.

8.7.5 Deadlock Recovery Strategies

8.7.5.1 1. Abort All Deadlocked Processes

- **Pros**:
  - Guarantees **immediate deadlock resolution**.
- **Cons**:
  - **Costly**: Significant loss of work if processes have completed substantial computation.

8.7.5.2 2. Abort Processes One at a Time

- **Pros**:
  - **Selective**: Minimizes work loss.
- **Cons**:
  - Requires **careful decision-making** to choose which process to terminate first.

8.7.5.3 Criteria for Process Termination When selecting processes to abort, consider the following factors: 1. **Process Priority**: - Lower-priority processes may be terminated first to minimize system impact. 2. **Execution Progress**: - **Avoid terminating processes close to completion** (high wasted effort). 3. **Resource Usage**: - Terminating processes with **fewer allocated resources** is more economical. 4. **Resource Requirements for Completion**: - Processes requiring **many additional resources** may be better candidates for termination. 5. **Number of Processes to Terminate**: - Prefer terminating the **fewest processes** possible to resolve the deadlock. 6. **Process Type (Interactive vs. Batch)**: - **Interactive processes** (e.g., user-facing) may have higher priority to remain running. - **Batch processes** (e.g., background tasks) may be safer to terminate.

8.7.6 Summary and Key Takeaways

- **Deadlock Detection Algorithm**:
  - Uses `Work`, `Finish`, `Allocation`, and `Request` data structures.
  - Iteratively checks for processes that can complete and release resources.
  - Detects deadlock if any `Finish[i] = false` remains.
- **Optimistic Assumption**:
  - Assumes processes will release resources soon; if incorrect, deadlocks are detected in subsequent runs.
- **Recovery Strategies**:
  - **Abort all** vs. **abort one at a time**, with trade-offs in cost and selectivity.
- **Termination Criteria**:
  - Prioritize minimizing work loss and system impact by considering process attributes (priority, progress, resource usage, etc.).

8.8 Mutual Exclusion and Hold & Wait

8.8.1 Introduction to Deadlock Prevention

- **Deadlock Definition**: A deadlock occurs when a set of processes are blocked indefinitely because each process holds a resource while waiting for another resource held by a different process.
- **Necessary Conditions for Deadlock**: Four conditions must exist **simultaneously** for a deadlock to occur:
  1. **Mutual Exclusion**
  2. **Hold and Wait**
  3. **No Preemption**
  4. **Circular Wait**
- **Deadlock Prevention Objective**: Strategies designed to **ensure the system never enters a deadlock state** by **breaking at least one of the four necessary conditions**.

8.8.2 Breaking the Mutual Exclusion Condition

8.8.2.1 Definition of Mutual Exclusion

- **Mutual Exclusion**: A condition where **certain resources cannot be used by more than one process simultaneously**.
  - Applies to **non-shareable resources** (e.g., printers, tape drives).
  - Does **not** apply to **shareable resources** (e.g., read-only files).

8.8.2.2 Types of Resources

1. **Non-Shareable Resources**
   - **Definition**: Resources that **must be used exclusively by one process at a time**.
   - **Example**: A printer.
     - If two processes attempt to print simultaneously, the output would be **garbled** (e.g., mixed content from both processes on the same page).
   - **Requirement**: **Mutual exclusion must be enforced** to prevent corruption or conflicts.
2. **Shareable Resources**
   - **Definition**: Resources that **can be accessed concurrently by multiple processes** without conflicts.
   - **Example**: Read-only files.
     - Multiple processes can read the same file simultaneously without interference.
   - **Requirement**: **No need for mutual exclusion**-processes can access these resources without waiting.

8.8.2.3 Strategy to Break Mutual Exclusion

- **Key Idea**: **Enforce mutual exclusion only for non-shareable resources**.
  - **Shareable resources** should **not** be subject to mutual exclusion.
  - **Processes never wait for shareable resources** since they can be accessed concurrently.
- **Implementation**:
  - Classify resources as **shareable** or **non-shareable**.
  - Apply mutual exclusion **only to non-shareable resources**.

8.8.3 Breaking the Hold & Wait Condition

8.8.3.1 Definition of Hold & Wait

- **Hold & Wait**: A condition where a process **holds at least one resource while waiting to acquire additional resources** held by other processes.
- **Prevention Goal**: Ensure that a process **never holds a resource while waiting for another**.

8.8.3.2 Two Protocols to Avoid Hold & Wait

8.8.3.2.1 Protocol 1: All-or-Nothing Resource Allocation

- **Requirement**: A process **must request and acquire all required resources before execution begins**.
  - **No partial allocation**-either all resources are granted, or none.
- **Advantages**:
  - Guarantees that a process **never holds some resources while waiting for others**.
- **Disadvantages**:
  - **Poor resource utilization**:
    - Resources may be **held idle** for long periods even if they are only needed later.
- **Example**:

- A process needs:
  1. A **DVD drive** (to read data).
  2. A **disk file** (to store sorted data).
  3. A **printer** (to print results).
- **Protocol 1 Approach**:
  * The process **requests all three resources (DVD, disk file, printer) at the start**.
  * It **holds all resources until completion**, even though the printer is only needed at the end.
  * **Result**: The printer remains **unnecessarily occupied** for most of the process's execution.

8.8.3.2.2 Protocol 2: Release-Before-Request

- **Requirement**: A process **can only request new resources when it holds none**.
  - If a process needs additional resources:
    1. It **must release all currently held resources**.
    2. Then, it can **request all required resources again**.
- **Advantages**:
  - **Better resource utilization** compared to Protocol 1.
  - Prevents long-term holding of resources not immediately in use.
- **Disadvantages**:
  - Still **less efficient** than dynamic allocation (where resources are acquired/released as needed).
  - May lead to **frequent resource requests and releases**, increasing overhead.
- **Example**:
  - Same process as above (DVD -> disk file -> printer).
  - **Protocol 2 Approach**:
    1. **First Phase**:
       * Requests **DVD drive and disk file**.
       * Copies data from DVD to disk and sorts it.
       * **Releases both resources** (DVD and disk file).
    2. **Second Phase**:
       * Requests **disk file and printer**.
       * Prints the sorted data.
       * **Releases both resources** and terminates.
  - **Result**:
    * The printer is **only held when needed** (unlike Protocol 1).
    * However, the disk file is **requested twice**, increasing overhead.

8.8.3.3 Comparison of Protocols

Aspect	Protocol 1 (All-or-Nothing)	Protocol 2 (Release-Before-Request)
Resource Allocation	All resources at once.	Resources in phases (release before new request).
Resource Utilization	Poor (resources held unnecessarily).	Better (resources released when not in use).
Overhead	Low (single request).	Higher (multiple requests/releases).
Deadlock Prevention	Effective (no hold & wait).	Effective (no hold & wait).

- **Conclusion**: Both protocols **prevent deadlock** by eliminating the hold & wait condition, but **at the cost of resource utilization efficiency**.

8.8.4 Summary of Key Concepts

8.8.4.1 Deadlock Prevention Strategies

- **Objective**: Ensure the system **never enters a deadlock state** by breaking **at least one** of the four necessary conditions.
- **Focus of This Lecture**: Breaking **mutual exclusion** and **hold & wait**.

8.8.4.2 Breaking Mutual Exclusion

- **Approach**: **Do not enforce mutual exclusion for shareable resources**.
  - **Non-shareable resources** (e.g., printers) **require** mutual exclusion.
  - **Shareable resources** (e.g., read-only files) **do not** require mutual exclusion.
- **Benefit**: Reduces unnecessary waiting for resources that can be accessed concurrently.

8.8.4.3 Breaking Hold & Wait

- **Two Protocols**:
  1. **All-or-Nothing Allocation**: Process requests **all resources upfront**.
     - **Pros**: Simple, no hold & wait.
     - **Cons**: Poor resource utilization.
  2. **Release-Before-Request**: Process **releases all resources before requesting new ones**.
     - **Pros**: Better utilization than Protocol 1.
     - **Cons**: Higher overhead due to repeated requests.
- **Trade-off**: Both methods **prevent deadlock** but **sacrifice efficiency**.

8.8.4.4 Final Observations

- **Resource Utilization**: Both protocols lead to **suboptimal resource usage** compared to dynamic allocation.
- **Next Steps**: Future lectures will cover breaking the **no preemption** and **circular wait** conditions.

End of Notes

8.9 No Preemption and Circular Wait

8.9.1 Introduction

- This lecture continues the discussion on **deadlock prevention schemes**, focusing on breaking:
  1. **No preemption condition**
  2. **Circular wait condition**
- Previously covered:
  - Breaking **mutual exclusion**
  - Breaking **hold and wait** conditions

8.9.2 Breaking the No Preemption Condition

8.9.2.1 Definition of No Preemption

- **No preemption condition**: Once a process acquires a resource, the system **cannot forcibly take it away** (preempt it).
- To prevent deadlocks, this condition must be **broken** by allowing preemption under specific protocols.

8.9.2.2 Protocol 1: Voluntary Release of All Held Resources

8.9.2.2.1 Mechanism

- If a process **P1** holds some resources and requests an **additional resource** that **cannot be immediately allocated**, then:
  - **P1 must release all currently held resources** (voluntary preemption).
  - The process **restarts later** when it can acquire:
    - Its **original resources** (previously held).
    - The **newly requested resource**.

8.9.2.2.2 Example Scenario

- **Process P1** holds **Resource A** and **Resource B**.
- P1 requests **Resource C**, which is held by **Process P2** and unavailable.
- Instead of waiting (risking deadlock), **P1 releases A and B**.
- P1 is **restarted only when it can regain A, B, and C simultaneously**.

8.9.2.2.3 Key Characteristics

- **Altruistic approach**: P1 "sacrifices" its resources to avoid deadlock.
- **Resources released** are added back to the **system's free resource pool**.
- P1 **cannot proceed** until it secures **all required resources** (old + new).

8.9.2.3 Protocol 2: Forced Preemption from Waiting Processes

8.9.2.3.1 Mechanism

- If a process **P1** requests a resource held by another process (**P2**), the system checks:
  1. **Is the requested resource available?**
     - If **yes**, allocate it to P1.
     - If **no**, proceed to step 2.
  2. **Is the process holding the resource (P2) waiting for additional resources?**
     - If **yes**, **preempt the resource from P2** and allocate it to P1.
     - If **no**, P1 must wait (no preemption occurs).

8.9.2.3.2 Example Scenario

- **P1** requests a resource held by **P2** and **P3**.
- The system checks:
  - Are **P2** and **P3** waiting for other resources?
    - If **yes**, preempt their resources and give them to **P1**.
    - If **no**, P1 waits.

8.9.2.3.3 Key Characteristics

- **Greedy approach**: P1 takes resources from waiting processes to avoid deadlock.
- **Preemption only occurs if the holding process is itself blocked** (waiting for more resources).
- Ensures **progress** by breaking circular dependencies.

8.9.3 Breaking the Circular Wait Condition

8.9.3.1 Definition of Circular Wait

- **Circular wait**: A set of processes {P1, P2, …, Pn} where:

  - P1 waits for a resource held by P2.
  - P2 waits for a resource held by P3.
  - …
  - Pn waits for a resource held by P1.
- This **cyclic dependency** leads to deadlock.

8.9.3.2 Prevention via Linear Ordering of Resource Types

8.9.3.2.1 Mechanism

- Assign a **unique integer** to each resource type using a **mapping function**:
  - **F: R -> N**, where:
    - **R** = set of resource types.
    - **N** = set of integers (e.g., 1, 2, 3, …).
- **Rule**: Processes **must request resources in strictly increasing order** of their assigned numbers.

8.9.3.2.2 Example Scenario

- Resources are numbered:
  - **R1 = 1** (e.g., Printer)
  - **R2 = 2** (e.g., Scanner)
  - **R3 = 3** (e.g., GPU)
- **Process P2** holds **R2 (Scanner)**.
  - Later, P2 **cannot request R1 (Printer)** because **1 < 2** (violates increasing order).
  - It can only request **R3 (GPU)** next (since 3 > 2).

8.9.3.2.3 Key Characteristics

- **Eliminates circular waits** by enforcing a **total order** on resource requests.
- **Prevents cycles** because a process can never "go backward" in resource numbers.
- **Does not restrict resource usage**-only the **order of requests**.

8.9.3.2.4 Implementation Considerations

- Requires **advance knowledge** of resource types and their numbering.
- Processes must **plan requests** to comply with the ordering.
- **Trade-off**: Simplicity in prevention vs. potential **underutilization** of resources if processes cannot request lower-numbered resources later.

8.9.4 Summary of Key Points

8.9.4.1 No Preemption Prevention

Protocol	Approach	Behavior
Protocol 1	Voluntary release	Process releases **all held resources** if a new request is blocked; restarts later with all required resources.
Protocol 2	Forced preemption	If a requested resource is held by a **waiting process**, preempt it and allocate to the requester.

8.9.4.2 Circular Wait Prevention

- **Method**: Impose a **linear order** on resource types (e.g., assign integers).
- **Rule**: Processes must request resources in **increasing numerical order**.
- **Effect**: Breaks cyclic dependencies by **disallowing "backward" requests**.

8.9.5 Conclusion

- Breaking **no preemption** and **circular wait** are two critical strategies in **deadlock prevention**.
- **No preemption** can be addressed via:
  - **Altruistic release (Protocol 1)**
  - **Greedy preemption (Protocol 2)**
- **Circular wait** is prevented by **enforcing a resource hierarchy** (linear ordering).
- These methods **do not eliminate deadlocks entirely** but **reduce their likelihood** by structuring resource allocation.

8.10 Recording of Operating Systems Week 7 - Live Session on 26-04-25

8.10.1 1. Introduction to CPU Scheduling

8.10.1.1 1.1 Overview of CPU Scheduling

- **Definition**: CPU scheduling is a process that allows one process to use the CPU while the execution of another process is on hold.
- **Ready Queue**: Processes waiting for CPU time reside in the **ready queue**.
- **Scheduler**: The **short-term scheduler** selects a process from the ready queue and allocates the CPU to it.

8.10.1.2 1.2 Role of the Dispatcher

- **Dispatcher Module**:
  - Gives control of the CPU to the process selected by the short-term scheduler.
  - Handles **context switching** (switching from one process to another).
  - Switches from **kernel mode** to **user mode** (required for security, memory management, and network operations).
  - Jumps to the correct location in the user program to restart execution.
- **Dispatch Latency**:
  - The time taken by the dispatcher to stop one process and start another.
  - **Example**: Time taken for one person to vacate an ATM and another to start using it.

8.10.1.3 1.3 When CPU Scheduling Occurs CPU scheduling is required in the following scenarios: 1. **Running -> Waiting State**: A process moves from running to waiting (e.g., I/O request). 2. **Running -> Ready State**: A process is preempted (e.g., time slice expires). 3. **Waiting -> Ready State**: A process completes I/O and moves to the ready queue. 4. **Termination**: A process completes execution.

8.10.1.4 1.4 Types of CPU Scheduling

- **Non-Preemptive Scheduling**:
  - A process holds the CPU until it voluntarily releases it (e.g., completes execution or moves to waiting state).
  - **Example**: A person using an ATM until their transaction is complete, regardless of how long it takes.
- **Preemptive Scheduling**:

- The OS can forcibly remove a process from the CPU (e.g., due to time slice expiration or higher-priority process arrival).
- **Example**: Forcing a person to leave the ATM if they are taking too long.

8.10.2 2. CPU Scheduling Criteria

8.10.2.1 2.1 Key Performance Metrics

1. **CPU Utilization**:
   - Goal: Keep the CPU as busy as possible (ideally 100%).
   - Real-world systems:
     - **Lightly loaded**: ~40% utilization.
     - **Heavily loaded**: ~90% utilization.
   - **Example**: ATMs display advertisements when idle to maximize utilization.
2. **Throughput**:
   - Number of processes completed per unit time (e.g., processes per second/hour).
3. **Turnaround Time**:
   - Time from process submission to completion.
   - **Formula**: `Turnaround Time = Completion Time - Arrival Time`.
4. **Waiting Time**:
   - Time a process spends waiting in the ready queue.
   - **Formula**: `Waiting Time = Turnaround Time - Burst Time`.
5. **Load Average**:
   - Average number of processes in the ready queue over a given time.
6. **Response Time**:
   - Time from process submission to the first response (e.g., first output).
   - **Formula**: `Response Time = Time when process first gets CPU - Arrival Time`.

8.10.3 3. CPU Scheduling Algorithms

8.10.3.1 3.1 First-Come, First-Served (FCFS)

8.10.3.1.1 3.1.1 Definition

- Processes are executed in the order they arrive in the ready queue.
- **Non-preemptive**: Once a process starts, it runs to completion.

8.10.3.1.2 3.1.2 Example Problem Given the following processes:

Process	Arrival Time	Burst Time
P1	0	2
P2	1	3
P3	5	3
P4	6	4

8.10.3.1.3 3.1.3 Gantt Chart Construction

1. **Time 0**: Only P1 is in the ready queue -> Execute P1 (0-2).
2. **Time 2**: P2 arrives (P1 completes). Execute P2 (2-5).
3. **Time 5**: P3 arrives. Execute P3 (5-8).
4. **Time 8**: P4 arrives. Execute P4 (8-12).

8.10.3.1.4 3.1.4 Calculations

Process	Completion Time	Turnaround Time	Waiting Time	Response Time
P1	2	2 - 0 = 2	2 - 2 = 0	0 - 0 = 0
P2	5	5 - 1 = 4	4 - 3 = 1	2 - 1 = 1
P3	8	8 - 5 = 3	3 - 3 = 0	5 - 5 = 0
P4	12	12 - 6 = 6	6 - 4 = 2	8 - 6 = 2

- **Average Waiting Time**: `(0 + 1 + 0 + 2) / 4 = 0.75`
- **Average Turnaround Time**: `(2 + 4 + 3 + 6) / 4 = 3.75`

8.10.3.1.5 3.1.5 Challenges of FCFS

- **Convoy Effect**: Long processes block shorter ones, leading to poor CPU utilization and high average waiting time.
- **Example**: A process with a 50-unit burst time holds up shorter processes in a billing queue.

8.10.3.2 3.2 Shortest Job First (SJF)

8.10.3.2.1 3.2.1 Definition

- The process with the **shortest burst time** is selected next.
- **Non-preemptive**: Once a process starts, it runs to completion.
- **Preemptive variant**: Shortest Remaining Time First (SRTF).

8.10.3.2.2 3.2.2 Example Problem (Non-Preemptive)

Given the following processes:

Process	Arrival Time	Burst Time
P1	1	3
P2	1	4
P3	1	2
P4	2	4

8.10.3.2.3 3.2.3 Gantt Chart Construction

1. **Time 0-1**: No process -> Idle.
2. **Time 1**: P1, P2, P3 arrive. Select P3 (shortest burst time) -> Execute P3 (1-3).
3. **Time 3**: P1 and P2 remain. Select P1 (burst time = 3) -> Execute P1 (3-6).
4. **Time 6**: P2 and P4 remain. Both have burst time = 4 -> Use FCFS (P2 arrived first) -> Execute P2 (6-10).
5. **Time 10**: Execute P4 (10-14).

8.10.3.2.4 3.2.4 Calculations

Process	Completion Time	Turnaround Time	Waiting Time
P3	3	3 - 1 = 2	2 - 2 = 0
P1	6	6 - 1 = 5	5 - 3 = 2
P2	10	10 - 1 = 9	9 - 4 = 5
P4	14	14 - 2 = 12	12 - 4 = 8

- **Average Waiting Time**: `(0 + 2 + 5 + 8) / 4 = 3.75`

8.10.3.2.5 3.2.5 Preemptive SJF (SRTF)

- **Strategy**: Execute processes in **1-unit time slices**, re-evaluating the shortest remaining time at each step.

8.10.3.2.6 3.2.6 Example Problem (SRTF)

Given the following processes:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

8.10.3.2.7 3.2.7 Gantt Chart Construction

1. **Time 0**: Execute P1 (0-1). Remaining burst time: P1=4.
2. **Time 1**: P2 arrives (burst time=3 < P1=4) -> Preempt P1, execute P2 (1-4).
3. **Time 4**: P2 completes. Remaining: P1=4, P3=8, P4=6 -> Execute P1 (4-8).
4. **Time 8**: P1 completes. Remaining: P3=8, P4=6 -> Execute P4 (8-14).
5. **Time 14**: Execute P3 (14-22).

8.10.3.2.8 3.2.8 Calculations

Process	Completion Time	Turnaround Time	Waiting Time
P2	4	4 - 1 = 3	3 - 3 = 0
P1	8	8 - 0 = 8	8 - 5 = 3
P4	14	14 - 3 = 11	11 - 6 = 5
P3	22	22 - 2 = 20	20 - 8 = 12

- **Average Waiting Time**: `(0 + 3 + 5 + 12) / 4 = 5`

8.10.3.3 3.3 Priority Scheduling

8.10.3.3.1 3.3.1 Definition

- Processes are selected based on **priority** (lower number = higher priority).
- Can be **preemptive** or **non-preemptive**.

8.10.3.3.2 3.3.2 Example Problem (Non-Preemptive)

Given the following processes:

Process	Arrival Time	Burst Time	Priority
P1	0	8	2
P2	1	3	1
P3	2	6	4
P4	3	2	3

8.10.3.3.3 3.3.3 Gantt Chart Construction

1. **Time 0**: Only P1 -> Execute P1 (0-8).
2. **Time 8**: P2, P3, P4 have arrived. Select P2 (highest priority=1) -> Execute P2 (8-11).
3. **Time 11**: Remaining: P3, P4. Select P4 (priority=3 < P3=4) -> Execute P4 (11-13).
4. **Time 13**: Execute P3 (13-19).

8.10.3.3.4 3.3.4 Calculations

Process	Completion Time	Turnaround Time	Waiting Time
P1	8	8 - 0 = 8	8 - 8 = 0
P2	11	11 - 1 = 10	10 - 3 = 7
P4	13	13 - 3 = 10	10 - 2 = 8
P3	19	19 - 2 = 17	17 - 6 = 11

- **Average Waiting Time**: `(0 + 7 + 8 + 11) / 4 = 6.5`

8.10.3.3.5 3.3.5 Starvation Problem

- Low-priority processes may **never execute** if high-priority processes keep arriving.
- **Solution**: **Aging** (gradually increase priority of waiting processes).

8.10.4 4. Exam Preparation Notes

8.10.4.1 4.1 Problem-Solving Approach

1. **Always start with arrival time** to determine the order of execution.
2. **For FCFS**: Execute processes in arrival order.
3. **For SJF/SRTF**: Select the process with the shortest (remaining) burst time.
4. **For Priority Scheduling**: Select the process with the highest priority (lowest number).
5. **Calculate metrics**:
    - **Completion Time**: When the process finishes.
    - **Turnaround Time**: `Completion Time - Arrival Time`.
    - **Waiting Time**: `Turnaround Time - Burst Time`.
    - **Response Time**: `First CPU Time - Arrival Time`.

8.10.4.2 4.2 Common Exam Questions

- **Numerical problems** (40-45 marks out of 50).
- **Gantt chart construction** for FCFS, SJF, SRTF, Priority.
- **Calculations** of average waiting time, turnaround time, and response time.
- **Preemptive vs. Non-preemptive** distinctions.

8.10.4.3 4.3 Key Terms to Remember

Term	Definition
Burst Time	Time required for a process to complete execution.
Arrival Time	Time when a process enters the ready queue.
Completion Time	Time when a process finishes execution.
Turnaround Time	`Completion Time - Arrival Time`.

Term	Definition
Waiting Time	`Turnaround Time - Burst Time`.
Response Time	Time from submission to first CPU allocation.
Convoy Effect	Long processes block shorter ones, reducing CPU utilization.
Dispatch Latency	Time taken to switch between processes.
Preemptive	OS can interrupt a running process.
Non-Preemptive	Process runs to completion without interruption.

8.11 Resource Allocation Graph Algorithm

8.11.1 1. Introduction to Resource Allocation in Deadlock Avoidance

- Deadlock avoidance algorithms differ based on the **type of resources** in the system:
  - **Single-instance resource types** -> Use **Resource Allocation Graph (RAG)**.
  - **Multiple-instance resource types** -> Use **Banker's Algorithm**.
- This lecture focuses on **deadlock avoidance using the Resource Allocation Graph (RAG)** for systems with **single-instance resource types**.

8.11.2 2. Resource Allocation Graph (RAG) Overview

8.11.2.1 2.1 Definition

- A **Resource Allocation Graph (RAG)** is a **graphical representation** of the **current state of a system**.
- It captures:
  - **Resource allocations** to processes.
  - **Resource requests** by processes.

8.11.2.2 2.2 Components of RAG

- **Vertices (Nodes):**
  - **Processes (P1, P2, …, Pn)** - Represented as circles (o).
  - **Resources (R1, R2, …, Rm)** - Represented as squares (□).
- **Edges (Connections):**
  1. **Assignment Edge (Allocation Edge)**
     - **Direction:** Resource (□) -> Process (o).
     - **Meaning:** The resource is **currently allocated** to the process.
     - **Example:** `R1 -> P1` means **R1 is allocated to P1**.
  2. **Request Edge**
     - **Direction:** Process (o) -> Resource (□).
     - **Meaning:** The process is **requesting** the resource.
     - **Example:** `P2 -> R2` means **P2 is requesting R2**.

8.11.3 3. Claim Edge: Extension for Deadlock Avoidance

8.11.3.1 3.1 Definition

- A **claim edge** is an **additional edge** introduced in the **deadlock avoidance algorithm**.
- **Representation:** Dashed line (——>).
- **Meaning:** Indicates that a process **may request** a resource **in the future** (but has not yet done so).

8.11.3.2 3.2 Behavior of Claim Edges

1. **Conversion to Request Edge:**
   - When a process **actually requests** the resource, the **claim edge** becomes a **request edge** (solid line).
2. **Conversion to Assignment Edge:**
   - If the request is **granted**, the **request edge** becomes an **assignment edge**.
3. **Reversion to Claim Edge:**
   - When the process **releases** the resource, the **assignment edge** reverts to a **claim edge**.

8.11.3.3 3.3 Key Requirement

- **Processes must declare their maximum resource requirements in advance.**
  - The system must know **all possible future requests** (claim edges) to perform deadlock avoidance.

8.11.4 4. Deadlock Avoidance Using RAG

8.11.4.1 4.1 Algorithm Workflow

1. **Initial State:**
   - The RAG includes **assignment edges** (current allocations) and **claim edges** (future possible requests).
2. **Request Handling:**
   - When a process **requests a resource**, the algorithm:
     - Converts the **claim edge** -> **request edge**.
     - Checks if converting the **request edge** -> **assignment edge would create a cycle** in the graph.
3. **Cycle Detection:**
   - **If no cycle:** The request is **granted** (safe state).
   - **If a cycle exists:** The request is **denied** (potential deadlock).

8.11.4.2 4.2 Example Scenario

8.11.4.2.1 System Setup:

- **Processes:** P1, P2
- **Resources:** R1, R2
- **Initial RAG:**
  - `P1 ----(claim)---> R2` (P1 may request R2 in the future).
  - `P2 ----(claim)---> R2` (P2 may request R2 in the future).

8.11.4.2.2 Case 1: P2 Requests R2

1. **Convert claim edge to request edge:**
   - `P2 -> R2` (solid request edge).
2. **Check if converting to assignment edge (`R2 -> P2`) creates a cycle:**
   - **Result:** A cycle is detected (e.g., `P1 -> R2 -> P2 -> R1 -> P1` if R1 is also involved).
   - **Decision: Request denied** (unsafe state).

8.11.4.2.3 Case 2: P1 Requests R2

1. **Convert claim edge to request edge:**
   - `P1 -> R2` (solid request edge).
2. **Check if converting to assignment edge (`R2 -> P1`) creates a cycle:**
   - **Result:** No cycle detected.

- **Decision: Request granted** (safe state).

8.11.5 5. Key Observations & Rules

1. **Single-Instance Limitation:**
   - RAG-based deadlock avoidance **only works for single-instance resources**.
   - For **multiple instances**, the **Banker's Algorithm** is used instead.
2. **Safety Condition:**
   - A system is in a **safe state** if **no cycles** exist in the RAG **after granting a request**.
3. **Claim Edge Importance:**
   - Claim edges **must be declared in advance** to predict future requests.
   - Without them, the algorithm **cannot guarantee deadlock avoidance**.
4. **Dynamic Edge Conversion:**
   - Edges **change type** based on process actions:
     - **Claim -> Request -> Assignment -> Claim** (upon release).

8.11.6 6. Summary of Steps for Deadlock Avoidance Using RAG

1. **Construct the initial RAG** with:
   - Assignment edges (current allocations).
   - Claim edges (future possible requests).
2. **When a process requests a resource:**
   - Convert the **claim edge** to a **request edge**.
   - Temporarily convert the **request edge** to an **assignment edge**.
   - **Check for cycles** in the modified graph.
3. **Decision Making:**
   - **If no cycle:** Grant the request (system remains in a safe state).
   - **If cycle exists:** Deny the request (prevents potential deadlock).
4. **Upon resource release:**
   - Convert the **assignment edge** back to a **claim edge**.

8.11.7 7. Conclusion

- The **Resource Allocation Graph (RAG) algorithm** is a **deadlock avoidance** technique for **single-instance resources**.
- It relies on:
  - **Graphical representation** of allocations and requests.
  - **Claim edges** to predict future resource needs.
  - **Cycle detection** to ensure safety.
- **Key Takeaway:** A system is **safe** if no cycles exist in the RAG after granting a request. If a cycle is detected, the request is **blocked** to prevent deadlock.

8.12 Resource Allocation Graph

8.12.1 1. Introduction to Resource Allocation Graph (RAG)

- **Purpose**:
  - A **Resource Allocation Graph (RAG)** is a **visual tool** used in **deadlock detection and prevention** in operating systems.
  - It helps **analyze and represent** the allocation of resources to processes.
  - Useful for **identifying potential deadlocks** by examining cycles in the graph.

8.12.2 2. Graph Representation Basics

8.12.2.1 2.1 Graph Definition

- A graph is defined as a set of **vertices (V)** and **edges (E)**.
  - **V (Vertices)** is partitioned into **two types**:
    1. **Processes (P)** - Represented by **circles (o)**.
    2. **Resources (R)** - Represented by **square boxes (□)**.
  - **Instances of a resource** are indicated by **small circles or squares inside the resource's box**.

8.12.2.2 2.2 Types of Edges

- **Directed edges** are used to represent relationships between processes and resources.
  - **Request Edge (P -> R)**:
    - Direction: **From a process (P) to a resource (R)**.
    - Meaning: The process **requests** an instance of the resource.
  - **Assignment Edge (R -> P)**:
    - Direction: **From a resource (R) to a process (P)**.
    - Meaning: The resource **has been allocated** to the process.

8.12.3 3. Example Analysis of Resource Allocation Graphs

8.12.3.1 3.1 Example 1: Simple Resource Allocation

- **Resources and Allocations**:
  - **R1 (Resource 1)**:
    - Has **3 instances**.
    - Allocated to: **P1, P2, P4**.
  - **R2 (Resource 2)**:
    - Has **1 instance**.
    - Allocated to: **P3**.
    - **P4** has **requested** an instance of R2.
  - **R3 (Resource 3)**:
    - Has **2 instances**.
    - **Not allocated or requested** by any process.
- **Deadlock Check**:
  - **Four necessary conditions for deadlock** (must exist simultaneously):
    1. **Mutual Exclusion** - At least one resource must be non-sharable.
    2. **Hold and Wait** - A process holds a resource while waiting for another.
    3. **No Preemption** - Resources cannot be forcibly taken from a process.
    4. **Circular Wait** - A circular chain of processes exists where each waits for a resource held by another.
  - **Observation**:
    - **No circular wait** exists in this graph -> **No deadlock**.
    - **Reason**: The edges do **not form a cycle** (they are not circular).

8.12.3.2 3.2 Example 2: Graph with a Cycle (Deadlock Present)

- **Graph Structure**:
  - **Edges**:
    - P0 -> R1 (request)
    - R1 -> P1 (allocation)

        - P1 -> R2 (request)
        - R2 -> P0 (allocation)
    - **Analysis**:
        - The edges form a **closed loop (cycle)**: **P0 -> R1 -> P1 -> R2 -> P0**.
        - **Conclusion**: A **deadlock exists** because of the **circular wait**.

8.12.3.3 3.3 Example 3: Graph without a Cycle (No Deadlock)

- **Graph Structure**:
    - Edges do **not** follow a circular path.
    - **Conclusion**: **No deadlock** because **no cycle exists**.

8.12.3.4 3.4 Example 4: Multiple Cycles in a Graph

- **Graph Structure**:
    - **Two cycles identified**:
        1. **Cycle 1**:
            - P1 -> R3 -> P2 -> R1 -> P1
        2. **Cycle 2**:
            - P1 -> R3 -> P2 -> R2 -> P4 -> R1 -> P1
    - **Key Observation**:
        - If a graph has **no cycles**, there is **no deadlock**.
        - If a graph has **one or more cycles**, further analysis is needed.

8.12.4 4. Deadlock Detection Using Cycles in RAG

8.12.4.1 4.1 Single-Instance Resources

- **Condition**:
    - If **each resource has only one instance**, then:
        - **A cycle in the graph is both necessary and sufficient for deadlock**.
        - **Presence of a cycle -> Deadlock exists**.
        - **Absence of a cycle -> No deadlock**.

8.12.4.2 4.2 Multiple-Instance Resources

- **Condition**:
    - If **a resource has multiple instances**, then:
        - **A cycle is a necessary but not sufficient condition for deadlock**.
        - **Possible scenarios**:
            1. **Cycle exists, but no deadlock** (if extra instances can break the cycle).
            2. **Cycle exists, and deadlock occurs** (if no free instances are available).
- **Example Analysis**:
    - **Graph with two cycles** (from Example 4):
        - **R1** has **3 instances**:
            - **2 allocated** (to P1 and P3).
            - **1 free instance** available.
        - **If the free instance is allocated to P4**:
            - **Cycle 2 breaks** because P4 acquires both needed resources.
            - P4 **completes execution** and **releases resources**.
            - **R2 and R1** can then be assigned to **P2**, eliminating the deadlock.
        - **Conclusion**:

· Even with a cycle, **deadlock may not occur** if extra instances can resolve the dependency.

8.12.5 5. Summary of Key Concepts

8.12.5.1 5.1 Representation in RAG

- **Processes**: Circles (o).
- **Resources**: Square boxes (□).
- **Instances**: Small circles/squares inside resource boxes.
- **Request Edge (P -> R)**: Process requests a resource.
- **Assignment Edge (R -> P)**: Resource allocated to a process.

8.12.5.2 5.2 Cycle Detection and Deadlock Analysis

Condition	Single-Instance Resources	Multiple-Instance Resources
No Cycle in Graph	No deadlock	No deadlock
Cycle in Graph	Deadlock exists	Deadlock **may or may not** exist

8.12.5.3 5.3 Final Takeaways

1. **No Cycle -> No Deadlock** (always true).
2. **Cycle with Single-Instance Resources -> Deadlock** (always true).
3. **Cycle with Multiple-Instance Resources -> Deadlock Possible but Not Guaranteed** (depends on available instances).

8.13 Single Instance of Each Resource Type

8.13.1 Introduction to Deadlock Detection

- **Deadlock Prevention vs. Deadlock Detection**:
  - **Prevention/Avoidance**: System is *prevented* from entering a deadlock state.
  - **Detection**: System is *allowed* to enter a deadlock, but an algorithm detects its presence.
    - Upon detection, a **recovery algorithm** is executed to resolve the deadlock.
  - **Key Difference**: Detection does not prevent deadlocks but identifies and resolves them after occurrence.
- **Algorithms for Detection**:
  - **Single-Instance Resource Types**: Use **Wait-for graphs** (directed graphs).
  - **Multiple-Instance Resource Types**: Use alternative algorithms (not covered in this lecture).

8.13.2 Wait-for Graph (WFG)

8.13.2.1 Definition and Purpose

- A **directed graph** used to detect deadlocks in:
  - Operating systems.
  - Relational databases.
- **Nodes**: Represent **only processes** (unlike Resource Allocation Graphs, which include both processes and resources).
- **Edges**:
  - A directed edge from **Pi -> Pj** indicates that **Pi is waiting for Pj to release a resource**.
  - **No self-loops** (a process cannot wait for itself).

8.13.2.2 Cycle Detection Principle

- **Deadlock Condition**:
  - A system is in deadlock **if and only if** the Wait-for graph contains **at least one cycle**.
  - **No cycle -> No deadlock**.
- **Algorithm Complexity**:
  - Cycle detection has a time complexity of **O(n2)**, where **n** = number of processes in the graph.
- **Periodic Invocation**:
  - The detection algorithm is **run periodically** to check for cycles.

8.13.3 Constructing a Wait-for Graph from a Resource Allocation Graph (RAG)

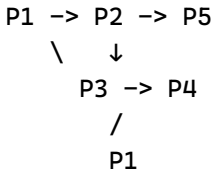
8.13.3.1 Step-by-Step Process

1. **Identify Processes**:
   - List all processes involved in resource requests.
2. **Identify Resources and Holders**:
   - List all resources and determine which process currently holds each.
3. **Determine Waiting Relationships**:
   - For each process, identify:
     - Which resources it is **waiting for**.
     - Which processes **hold those resources**.
4. **Draw Directed Edges**:
   - For each waiting relationship, draw an edge from the **waiting process** to the **holding process**.

8.13.3.2 Example Construction Given Resource Allocation Graph (RAG): - Processes: P1, P2, P3, P4, P5. - Resource requests: - P1 requests **R1** (held by P2). - P2 requests **R3** (held by P5) and **R4** (held by P3). - P3 requests **R5** (held by P4). - P4 requests **R2** (held by P1).

Corresponding Wait-for Graph (WFG) Edges: 1. **P1 -> P2** (P1 waits for P2 holding R1). 2. **P2 -> P5** (P2 waits for P5 holding R3). 3. **P2 -> P3** (P2 waits for P3 holding R4). 4. **P3 -> P4** (P3 waits for P4 holding R5). 5. **P4 -> P1** (P4 waits for P1 holding R2).

Resulting Wait-for Graph:



8.13.4 Cycle Detection and Deadlock Identification

8.13.4.1 Detected Cycles in the Example

1. **First Cycle**:
   - **P1 -> P2 -> P4 -> P1**.
2. **Second Cycle**:
   - **P1 -> P2 -> P3 -> P4 -> P1**.

8.13.4.2 Interpretation

- **Presence of Cycles**:

- Both cycles confirm that the system is in a **deadlock state**.
- Processes in the cycle are **mutually waiting** for each other's resources indefinitely.

8.13.5 Summary of Wait-for Graph Properties

8.13.5.1 Key Characteristics

- **Nodes**: Only processes (no resources).
- **Edges**: Represent **waiting relationships** between processes.
- **Cycle ≡ Deadlock**:
  - A cycle in the WFG is a **necessary and sufficient condition** for deadlock in single-instance resource systems.

8.13.5.2 Construction Steps Recap

1. **List all processes** involved in resource requests.
2. **Map resources to holding processes**.
3. **Identify waiting processes** for each resource.
4. **Draw edges** from waiting to holding processes.
5. **Run cycle detection** to check for deadlocks.

8.13.5.3 Practical Implications

- **Efficiency**: Cycle detection is **polynomial-time (O(n2))**, making it feasible for real-time systems.
- **Periodic Checks**: The algorithm must be **invoked regularly** to ensure timely deadlock detection.
- **Recovery**: Upon detection, a **recovery mechanism** (e.g., process termination, resource preemption) is triggered.

8.14 Summary

8.14.1 1. Definition of Deadlock

- A **deadlock** is a situation in computing where a **set of processes** is **unable to proceed** because:
  - Each process is **waiting for a resource** that is **held by another process** in the set.
  - This creates a **circular dependency**, preventing any process from making progress.

8.14.2 2. Necessary Conditions for Deadlock

Four conditions **must hold simultaneously** for a deadlock to occur:

1. **Mutual Exclusion**
   - At least one resource must be **non-sharable** (only one process can use it at a time).
   - Example: A printer can only be used by one process at a time.
2. **Hold and Wait**
   - A process must be **holding at least one resource** while **waiting to acquire additional resources** held by other processes.
3. **No Preemption**
   - Resources **cannot be forcibly taken away** from a process.
   - They must be **released voluntarily** by the process holding them.
4. **Circular Wait**
   - A **circular chain** of processes exists where each process is waiting for a resource held by the next process in the chain.

8.14.3 3. Deadlock Handling Techniques

Operating systems employ different strategies to **manage deadlocks**:

8.14.3.1 3.1 Deadlock Prevention

- **Goal**: Modify the system design to **ensure that at least one of the four necessary conditions for deadlock cannot hold**.
- **Methods**:
  - **Eliminate Mutual Exclusion**: Allow sharable resources (not always possible).
  - **Eliminate Hold and Wait**: Require processes to request **all resources at once** (before execution begins).
  - **Allow Preemption**: Forcefully take resources from processes if needed.
  - **Break Circular Wait**: Impose a **total ordering** on resource types and require processes to request resources in increasing order.

8.14.3.2 3.2 Deadlock Avoidance

- **Goal**: Dynamically examine the **resource allocation state** to ensure that a **circular wait condition can never exist**.
- **Algorithms**:
  1. **Resource Allocation Graph (RAG) Algorithm**
     - Used for systems with **single-instance resource types**.
     - Constructs a **directed graph** where:
       - **Nodes** represent processes and resources.
       - **Edges** represent resource requests and allocations.
     - A **cycle in the graph** indicates a potential deadlock.
  2. **Banker's Algorithm**
     - Used for systems with **multiple instances of resource types**.
     - Determines whether a resource allocation request can be granted **without leading to a deadlock**.
     - Uses **safe state** analysis to ensure system remains deadlock-free.
- **Key Difference from Prevention**:
  - Prevention **eliminates one of the four conditions** statically.
  - Avoidance **dynamically checks** if granting a request could lead to a deadlock.

8.14.3.3 3.3 Deadlock Detection and Recovery

- **Goal**: Allow the system to **enter a deadlock state**, **detect it**, and then **recover**.
- **Detection Methods**:
  - **Wait-for Graph (Single-Instance Resources)**
    - A **simplified graph** that only includes **processes**.
    - An edge from **P1 -> P2** means P1 is waiting for a resource held by P2.
    - A **cycle** in the graph indicates a deadlock.
  - **Algorithm for Multiple-Instance Resources**
    - Similar to the **Banker's Algorithm** but used for detection rather than avoidance.
- **Recovery Methods**:
  1. **Process Termination**
     - **Abort all deadlocked processes** (simplest but costly).
     - **Abort one process at a time** until the deadlock is resolved.
  2. **Resource Preemption**
     - **Forcefully take resources** from processes and allocate them to others.

- May require **rollback** (restoring a process to a previous state).

8.14.3.4 3.4 Deadlock Ignorance (Ostrich Algorithm)

- **Used in systems where deadlocks are rare**.
- **Approach**: **Do nothing** and let the application handle deadlocks.
- **Justification**: The overhead of prevention/avoidance/detection may outweigh the cost of occasional deadlocks.

8.14.4 4. Summary of Key Algorithms

Algorithm	Purpose	Resource Type
Resource Allocation Graph	Deadlock detection (single instance)	Single-instance resources
Banker's Algorithm	Deadlock avoidance (multiple instances)	Multiple-instance resources
Wait-for Graph	Deadlock detection (simplified)	Single-instance resources

8.14.5 5. Practical Considerations

- **Prevention vs. Avoidance vs. Detection**:
  - **Prevention** is **conservative** (may reduce system efficiency).
  - **Avoidance** requires **runtime checks** (overhead but more flexible).
  - **Detection and Recovery** is **reactive** (useful when deadlocks are infrequent).
- **Ostrich Algorithm** is **cost-effective** in systems where deadlocks are rare.

8.14.6 6. Conclusion

- Deadlocks are a **critical issue** in operating systems.
- Different strategies (**prevention, avoidance, detection, ignorance**) are used based on system requirements.
- The choice of technique depends on **resource types, system overhead, and deadlock frequency**.

8.15 System Model

8.15.1 1. Introduction to Deadlocks in Multi-Programming Environments

- In a **multi-programming environment**, multiple processes execute concurrently, competing for a **limited number of resources**.
- **Resources** may include:
  - CPU cycles
  - Main memory (RAM)
  - Disk storage
  - Files
  - I/O devices (printers, monitors, DVD drives)
  - Network connections

8.15.1.1 1.1 Resource Request and Allocation Mechanism

- A **process** must **request** a resource before using it.
- If the resource is **unavailable**, the process **waits** until it becomes free.
- After using the resource, the process must **release** it so others can access it.
- **Key System Calls for Resource Management**:

- **I/O Devices**: `request()` and `release()`
- **Files**: `open()` and `close()`
- **Memory**: `allocate()` and `free()`

8.15.1.2 1.2 Definition of Deadlock

- A **deadlock** is a state in an operating system where **no progress** can be made-processes are **permanently blocked**.
- Occurs when a **set of blocked processes** each hold a resource and **wait indefinitely** for another resource held by a different process in the same set.

8.15.2 2. Real-World Analogies of Deadlock

Deadlocks are not just a computing concept-they appear in real-life scenarios where **circular dependencies** prevent progress.

8.15.2.1 2.1 Financial Deadlock

- **"It takes money to make money."**
  - To **invest** and generate more money, you need **initial capital**.
  - Without money, you **cannot start**, leading to a **standstill**.

8.15.2.2 2.2 Employment Deadlock

- **"You can't get a job without experience, and you can't get experience without a job."**
  - **Job seekers** need **experience** to be hired.
  - **Employers** require **prior experience** before hiring.
  - Results in a **vicious cycle** where neither condition can be satisfied.

8.15.2.3 2.3 Traffic Deadlock (Railway Crossing Example)

- **Scenario**: People from all sides of a railway track wait for others to move first.
- **Outcome**: Everyone **blocks each other**, leading to **no movement**.
- **Key Takeaway**: Demonstrates how **mutual waiting** leads to a **system-wide halt**.

8.15.3 3. Formal Definition and Conditions for Deadlock

8.15.3.1 3.1 Formal Definition

- A deadlock occurs when:
  - A **set of processes** are **blocked** because each process is **holding a resource** and **waiting for another resource** held by a different process in the same set.
- **Example**:
  - **Process P1** holds **Resource R1** and waits for **Resource R2** (held by **P2**).
  - **Process P2** holds **Resource R2** and waits for **Resource R1** (held by **P1**).
  - **Result**: Both processes **wait indefinitely**.

8.15.3.2 3.2 Disk Copying Deadlock Example

- **Resources**: Two disks, **D1** and **D2**.
- **Processes**: **P1** and **P2**.
  - **P1** wants to copy **D1 -> D2**.

  - **P2** wants to copy **D2 -> D1**.
- **Execution Flow**:
  1. **P1** acquires **D1**.
  2. **P2** acquires **D2**.
  3. **P1** requests **D2** (held by **P2**) -> **blocks**.
  4. **P2** requests **D1** (held by **P1**) -> **blocks**.
- **Outcome**: **Deadlock**-neither process can proceed.

8.15.4 4. Root Causes of Deadlock

8.15.4.1 4.1 Finite Resources

- Deadlocks arise due to **limited resources** in a system.
- **Examples of Limited Resources**:
  - **Memory space** (RAM)
  - **CPUs** (number of processors)
  - **Files** (maximum open files)
  - **I/O Devices** (printers, monitors, DVD drives)

8.15.4.2 4.2 Resource Types vs. Instances

- **Resource Type**: A **category** of resources (e.g., **printer**).
- **Resource Instance**: A **specific unit** of that type (e.g., **Printer 1, Printer 2, Printer 3**).
- **Example**:
  - A system has **3 printers** -> **Resource Type = Printer**, **Instances = 3**.

8.15.4.3 4.3 Resource Usage Protocol

- A process **must** follow this sequence:
  1. **Request** the resource.
  2. **Use** the resource.
  3. **Release** the resource.
- **Violation** (e.g., holding a resource indefinitely) can lead to **resource starvation** and **deadlocks**.

8.15.5 5. Summary of Key Concepts

8.15.5.1 5.1 What is a Deadlock?

- A **permanent blockage** in a set of processes due to **circular waiting** for resources.

8.15.5.2 5.2 Why Do Deadlocks Occur?

- **Finite resources** + **improper resource management** (holding resources while waiting for others).

8.15.5.3 5.3 Real-World vs. Computing Deadlocks

Real-World Example	Computing Equivalent
Need money to make money	Process waits for a resource held by another
No job without experience	Process blocked due to circular dependency
Railway crossing gridlock	Processes holding and waiting for resources

8.15.5.4 5.4 Importance of Deadlock Handling

- **Prevents system freeze**: Ensures processes do not remain stuck indefinitely.
- **Optimizes resource usage**: Helps the OS manage limited resources efficiently.
- **Improves system reliability**: Reduces crashes and unresponsive states.

8.15.6 6. Preview of Next Session

- **Upcoming Topic**: **Handling Deadlock Scenarios**
  - **Detection** (identifying deadlocks)
  - **Prevention** (designing systems to avoid deadlocks)
  - **Recovery** (resolving deadlocks when they occur)
  - **Avoidance** (dynamic resource allocation to prevent deadlocks)